

Annabel Turner, Lucas Neto Garcia, Khuma Sharneck

Module Code: CS1DB

Assignment report Title: Design and Implementation of a Tennis Results Relational Database

Date (when the work completed): completed by 10.05.26

Actual hrs spent for the assignment: approximately 15-20 hours per student

Which Artificial Intelligence tools used (if applicable): ChatGPT used critically to support drafting, checking structure, and improving clarity.

1. INTRODUCTION



This assignment focuses on the design and implementation of a relational database for a professional tennis results system. The database stores structured information about players, coaches, tournaments, yearly tournament editions, matches, match participants and set scores. It supports men's and women's singles and doubles results from major professional tournaments, including Grand Slam events.

A relational database is suitable because the scenario contains clear relationships between entities. For example, one tournament can have many editions, one edition can contain many matches, and each match can involve multiple players. Storing this data in linked tables reduces redundancy, improves consistency and maintains data integrity through primary and foreign keys.

The system also supports analysis of tennis performance by storing match results, set scores and player-coach history. This allows SQL queries to investigate tournament participation, player performance, coaching changes and possible unusual performance improvements.

The project follows the database development process taught in CS1DB: creating an E-R model, converting it into a relational design, implementing it in PostgreSQL, inserting test data, validating the tables, and running SQL queries to demonstrate that the database works correctly.

2. DESIGN AND IMPLEMENT THE DATABASE

2.1 ER MODELLING

2.1.1 PURPOSE OF THE E-R MODEL

Figure 2.1 shows the Entity Relationship model created for the tennis results database. The purpose of the E-R model is to define the database structure before PostgreSQL implementation. It models the main entities required by the scenario, including players, coaches, tournaments, yearly editions, matches, participants and set scores.

The design supports men's and women's singles and doubles events, player-coach history and later analysis of unusual performance improvements. It also identifies the attributes, data types, primary keys, foreign keys, relationships, constraints and assumptions needed to build the database.

2.1.2 ENTITY AND ATTRIBUTE IDENTIFICATION

The E-R model contains eight main entities. The full entity list, attributes, primary keys, foreign keys and constraints are shown in **Appendix Figure 2.1.2.a**.

Entity	Main purpose
Player	Stores professional tennis player details.
Coach	Stores coach details and links coaches to former player records.
Tournament	Stores stable tournament details such as name, location and surface.
Tournament_Edition	Stores year-specific tournament details.
Match	Stores match details for a specific tournament edition.
Match_Participant	Links players to matches and resolves the many-to-many relationship.
Match_Set_Score	Stores each set score separately for a match.
Player_Coach_History	Stores current and previous coaching relationships over time.

Each entity was designed to store one main type of information, supporting normalisation and reducing repeated data. Primary keys uniquely identify records, while foreign keys maintain relationships between entities.

2.1.3 RELATIONSHIPS AND CARDINALITY

The E-R model shows how the entities are connected using relationships and cardinality. Cardinality identifies how many records in one table can relate to records in another table. The main relationships are shown below.

Relationship	Cardinality	Explanation
Tournament → Tournament_Edition	One-to-many	One tournament can have many yearly editions, but each edition belongs to one tournament.
Tournament_Edition → Match	One-to-many	One tournament edition can contain many matches, but each match belongs to one edition.
Match → Match_Participant	One-to-many	One match can have multiple participant records, allowing singles and doubles matches to be represented.
Player → Match_Participant	One-to-many	One player can appear in many match participant records.
Match → Match_Set_Score	One-to-many	One match can have many set score records, with each set stored separately.
Player → Player_Coach_History	One-to-many	One player can have multiple coach history records over time.
Coach → Player_Coach_History	One-to-many	One coach can appear in multiple coaching history records.
Player → Coach	One-to-zero-or-one / one-to-many	The former_player_id foreign key links each coach to a player record, reflecting that coaches must have professional playing experience.

The many-to-many relationship between Player and Match is resolved using Match_Participant. This allows one player to take part in many matches and one match to include multiple players, while also supporting both singles and doubles through side_no.

The relationship between Player and Coach is handled using Player_Coach_History, which stores start_date and end_date. This allows the database to record both current and previous coaching relationships instead of storing only one coach directly in the Player table. ✓

Overall, the relationships use primary and foreign keys to maintain referential integrity, reduce redundancy and keep the database consistent.

2.1.4 CONSTRAINTS AND ASSUMPTIONS

The E-R model uses constraints to keep the data valid and consistent. Primary keys such as player_id, match_id, tournament_id, coach_id and history_id uniquely identify records, while foreign keys link related tables and maintain referential integrity.

Domain constraints are used where values must be limited. For example, match_type is limited to singles or doubles, gender_category is limited to men or women, and winner_side / side_no must be either 1 or 2. Set numbers and game scores must also be valid, and date ranges must be logical. ✓

The main assumptions are that the database stores professional men's and women's tennis events, including singles and doubles matches. Each match belongs to one tournament edition and has two sides. Coaches are assumed to be former professional players, represented by the former_player_id foreign key in Coach. ✓

Doping concerns are not stored directly as a fixed attribute; instead, they can be identified by analysing match and set-score data for unusual performance improvements. These constraints and assumptions define the scope of the database and support accurate data storage. ✓

2.1.5 JUSTIFICATION OF DESIGN DECISIONS

The E-R model was designed to reduce repetition, maintain consistency and support the required tennis queries. Each entity represents one main subject, helping the database avoid duplicated data. ✓

Tournament and Tournament_Edition were separated because the same tournament occurs each year. This allows stable tournament details to be stored once, while year-specific dates are stored separately.

Match_Participant resolves the many-to-many relationship between players and matches and also supports singles and doubles by assigning players to sides. Match_Set_Score was separated from Match so that each set can be stored as a separate record rather than using repeated score columns. Player_Coach_History records coaching changes over time using start_date and end_date, while Coach links to Player through former_player_id to reflect that coaches must have professional playing experience. ✓

Overall, the design supports tournament tracking, match results, coach history and performance analysis while reducing redundancy.

2.1.6 THIRD NORMAL FORM JUSTIFICATION

The E-R model was designed to follow Third Normal Form (3NF) to reduce redundancy, improve consistency, and avoid update, insertion and deletion anomalies.

The model satisfies 1NF because attributes store single values and repeating groups are avoided. For example, each set score is stored as a separate record in Match_Set_Score rather than storing multiple set scores inside the Match table. The model satisfies 2NF because non-key attributes depend on the whole primary key. For example, in Match_Participant, attributes such as side_no depend on the combination of match_id and player_id, as they describe a specific player's role in a specific match. The model satisfies 3NF because non-key attributes do not depend on other non-key attributes. Each table stores data about one subject: player details are stored in Player, tournament details in Tournament, yearly tournament data in Tournament_Edition, and set scores in Match_Set_Score.

The design also avoids transitive dependencies. For example, tournament details such as location and surface_type are not stored in Match, and player details such as nationality are not stored in Match_Participant. This means changes only need to be made once in the correct table, supporting data integrity.

2.2 DATABASE IMPLEMENTATION IN POSTGRESQL

2.2.1 DATABASE SETUP ENVIRONMENT

The database was implemented using PostgreSQL through pgAdmin 4 in the university remote lab environment, accessed via Parallels Client. Appendix Figures 2.2.1.a and 2.2.1.b show the remote lab login and pgAdmin environment used to create, manage and query the tennis database.

2.2.2 SQL-DDL TABLE CREATION

The database was implemented using a normalised relational structure based on the E-R model developed in Section 2.1. Each table represents a single entity or relationship, such as player, coach, tournament, match, match_participant, and player_coach_history. This supports Third Normal Form by reducing repeated data and ensuring that attributes depend on the relevant primary key.

Relationships between entities were implemented using foreign keys to maintain referential integrity. For example, the coach table references the player table through former_player_id, enforcing the requirement that all coaches must have professional playing experience. Many-to-many relationships were resolved using associative tables, such as match_participant, which links players to matches without storing repeated player or match data.

The player_coach_history table was also included to record coaching relationships over time. This allows previous and current coaches to be stored using start_date and end_date, rather than storing only one current coach in the player table.

Overall, the implemented table structure reflects the earlier E-R diagram and ensures consistency between the conceptual design and the PostgreSQL implementation.



2.2.3 VALIDATION OF ENTITY CREATION

The entity creation stage was validated by checking that the PostgreSQL tables matched the structure defined in the E-R model. The created database includes the required base entities, including player, coach, tournament, and match, as well as linking entities such as match_participant and player_coach_history to support many-to-many and time-based relationships.

The screenshots in Appendix Figures **2.2.3.a–2.2.3.g** show that the tables were successfully created in pgAdmin and that the expected attributes, primary keys, foreign keys and data types are present. This supports entity integrity, as each table has a primary key to uniquely identify records, and referential integrity, as foreign keys are used to link related tables.

For example, the match_participant table validates the many-to-many relationship between players and matches, while the player_coach_history table validates the historical relationship between players and coaches by recording coaching periods using start_date and end_date.

Overall, the PostgreSQL implementation reflects the E-R model and demonstrates that the database entities were created correctly before data population was carried out.



2.2.4 SQL-DML DATA POPULATION

SQL-DML was used to populate the database after the tables had been created using SQL-DDL. The insert_data.sql script contained INSERT INTO statements for the base entities, including player, coach, tournament and match, as well as linking tables such as match_participant, match_set_score and player_coach_history. An example of the SQL-DML population process is shown in Appendix Figure 2.2.4.a. The populated table outputs are also shown in Appendix Figures 2.2.3.a–2.2.3.g.



2.2.5 VALIDATION OF DATA POPULATION

Data population was validated using SELECT statements and record-count checks. Appendix Figure 2.2.5.a shows that the key tables contain inserted records after the SQL-DML script was executed. The populated outputs in Appendix Figures 2.2.3.a–2.2.3.g provide further evidence that records were inserted into both base entities and linking tables. Overall, the database was populated correctly and was ready for SQL query testing in Section 2.3.



2.2.6 TESTING AND VALIDATION STRATEGY

Testing was carried out to check that the PostgreSQL implementation matched the E-R model and contained suitable data for query testing. This included checking table creation, SQL-DDL execution, constraints, and

populated relationship data. Appendix Figures 2.2.6.a–2.2.6.e provide evidence that the required tables were created, the SQL script ran successfully, key constraints were present, and related records could be retrieved using SELECT queries. These checks confirmed that the database was ready for final query testing.

Test Area	Purpose	Evidence Used
Entity creation	Check all required tables were created	Appendix Figure 2.2.6.a
SQL-DDL execution	Check the table creation script ran successfully	Appendix Figure 2.2.6.b
Constraint validation	Check primary keys, foreign keys, columns and constraints	Appendix Figure 2.2.6.c
Data population/query readiness	Check inserted records can be retrieved from related tables	Appendix Figures 2.2.6.d–2.2.6.e

2.3 QUERY DATA FROM THE DATABASE

SQL-DML SELECT queries were used to demonstrate that the database could retrieve information from both individual tables and related tables. Two simple queries and two complex queries were selected, with further query evidence included in the appendix.

2.3.1/2.3.2 SIMPLE AND COMPLEX QUERIES

The simple queries tested basic retrieval and aggregation from individual tables. Appendix Figure 2.3.1.a shows a WHERE query retrieving players from a specified nationality. Appendix Figure 2.3.1.b shows a grouping query using COUNT, GROUP BY and ORDER BY to summarise players by nationality. The complex queries tested retrieval across linked tables. Appendix Figure 2.3.2.a shows a match wins query using joins between match, match_participant and player. Appendix Figure 2.3.2.b shows a full match scoreline query using joins between match, tournament_edition, tournament and match_set_score.

2.3.3 QUERY VALIDATION

The SQL queries were executed in pgAdmin and checked against the expected results. The simple queries confirmed that individual tables could be filtered and summarised correctly, while the complex queries confirmed that related tables could be joined to return match, tournament, set-score and player performance data.

Overall, the results show that the database can return accurate information from both single tables and linked tables, supporting tennis performance analysis.

Query Test	Type	Expected Result	Actual Result
Retrieve female players	Simple	Only players with gender recorded as Women are returned	Passed — correct filtered records returned
Count total players	Simple	The total number of records in the player table is returned	Passed — total player count returned

Match wins per player	Complex	Players are listed with the number of match wins	Passed — correct win totals returned
Match scorelines	Complex	Match, tournament and set-score data are combined	Passed — full results returned

3. REFLECTION ON THE DESIGN AND IMPLEMENTATION OF THE DATABASE

3.1 CRITICAL EVALUATION USING RELATIONAL DATABASE PRINCIPLES

The database applies key relational database principles, including normalisation, referential integrity and the resolution of many-to-many relationships. The design follows 3NF by separating data into clear entities such as Player, Tournament, Match and Coach, reducing redundancy and improving consistency.

Linking tables are used effectively. Match_Participant resolves the many-to-many relationship between players and matches, while Player_Coach_History records coaching relationships over time. This makes the design more flexible than storing all data in a single table.

However, there are some limitations. Match_Participant uses side_no to represent singles and doubles matches, but it does not explicitly model doubles teams. A future improvement would be to add a Team entity for clearer doubles pairings. The Coach table also assumes every coach is a former player through former_player_id, which may reduce flexibility. Indexing frequently queried fields such as player_id, match_id and tournament_id would also improve performance as the database grows.

Overall, the database is well structured for the coursework scenario, but could be improved by making it more scalable and flexible.

3.2 LEGAL, SOCIAL, ETHICAL AND PROFESSIONAL ISSUES

The database stores personal data such as player names, nationalities and dates of birth, so a real system would need to follow GDPR principles, including secure storage, limited access, clear purpose and avoiding unnecessary data retention. There are also ethical issues around identifying possible doping concerns. Unusual performance improvements should not be treated as proof of doping, as this could create unfair assumptions or reputational harm. Such analysis should only be used as supporting evidence alongside proper investigation. Security measures such as authentication, authorisation, encryption, backups and recovery procedures would help protect the database from unauthorised access, data loss or misuse.

4. CONCLUSION

This project designed and implemented a relational database for tracking professional tennis players, tournaments, matches, set scores and player-coach relationships. The E-R model identified the main entities, attributes, keys, relationships, constraints and assumptions needed to meet the scenario requirements.

The database was implemented in PostgreSQL using SQL-DDL and populated using SQL-DML. SQL queries demonstrated that the database can retrieve information from individual and related tables. Overall, the database meets the main coursework requirements and provides a structured basis for analysing tournament results, coaching history and tennis performance.